Relatório Técnico: Base de Conhecimento, Protótipo de RAG e Telemetria

RES IA ELD UNB — Turma 2 | Grupo 7 — Orquestrador de Rolês (Brasília/DF)

Equipe: Letícia Miti Takahashi (Engenharia de Dados & RAG) e Maria Eduarda (DevOps & LLMOps)

Aplicação & RAG Online: orquestrador-roles.onrender.com/rag

Central Interativa de Entregáveis: https://orquestrador-roles.onrender.com/entregaveis

Acesso Interativo: Todos os artefatos (relatório, base 2FA de 93 rolês, quarentena e vetores) podem ser navegados e auditados interativamente no link acima, sem necessidade de baixar arquivos pesados.

1. Resumo Executivo e Checklist dos Artefatos

O objetivo deste entregável da Semana 2 é construir e auditar o mecanismo de **RAG (*Retrieval-Augmented Generation*)** fundamentado em base de conhecimento curada do DF, assegurando alta precisão na recuperação semântica, respostas sem alucinação e rastreabilidade total das fontes.

Requisito do Edital	Status	Localização no Repositório	Resumo Técnico
1. Base curada	**Concluído**	`data/banco_aprovado_2fa.json`	93 estabelecimentos aprovados pelo protocolo 2FA.
2. Mínimo 20 fontes	**Superado**	`data/banco_aprovado_2fa.json` (93 fontes)	164 vídeos auditados de 2026 (`@rolezandoembrasilia`).
3. Estratégia chunking	**Concluído**	Seção 2 + `src/rag/banco_vetorial.py`	*Entity-Centric Semantic Chunking* (1 chunk por local).
4. Embeddings	**Concluído**	Seção 3 + `src/rag/banco_vetorial.py`	`text-embedding-3-large` (3.072-d) + Cosseno.
5. Banco vetorial	**Concluído**	`data/banco_vetorial_embeddings.js`	93 vetores persistidos com busca top-K ($K = 3$).
6. Demonstração RAG	**Concluído**	`src/rag/prototipo_rag.py` + `.txt`	4 passos: Consulta → Chunks → Resposta → Fontes.
7. Diagrama & Pipeline	**Concluído**	Seções 4 e 5 deste relatório	Fluxos de Ingestão 2FA e Arquitetura de Runtime RAG.
8. Telemetria LLMOps	**Concluído**	Seção 6 + `src/telemetria/telemetria.py`	Langfuse Cloud v5.x (Auditoria RNF-03 e RNF-14).

2. Estratégia de Chunking (*Entity-Centric Semantic Chunking*)

2.1. Por que não utilizar *Fixed-Size Chunking*?

Vídeos curtos de gastronomia e lazer (30 a 60 segundos) apresentam narrativa não linear: o **nome do local** é citado na abertura, as **vibes** no meio e o **preço e endereço** nos segundos finais. Um particionamento cego por janela de caracteres separaria o endereço e o preço do nome, gerando fragmentação e induzindo alucinações durante a síntese.

2.2. Solução Adotada: 1 Documento-Fonte = 1 Chunk Semântico Autocontido

Cada estabelecimento aprovado consolida 4 camadas de informação validadas em um único bloco coeso (~ 120 a 190 tokens), garantindo que o vetor represente a totalidade semântica do rolê:

1. **Identidade e Categoria (Fator 1):** Nome oficial e classificação (`Restaurante`, `Bar/Pub`, `Cafeteria`, `Ao Ar Livre`).
2. **Ancoragem Geoespacial e Financeira (Fator 2):** Região Administrativa (RA), endereço validado via GPS Nominatim e preço médio em R$.
3. **Atmosfera e Cultura (Fator 1):** Tags de *Vibes* (`Romântico`, `Calmo`, `Baratinho`, `Kid Friendly`, `Animado`) e Resumo Sensorial.
4. **Evidência Oral Bruta (Whisper):** Trecho literal da transcrição de áudio do vídeo, preservando itens do cardápio e atrações.

Template Estruturado do Chunk Indexado no Banco Vetorial

```
Local: {nome} | Categoria: {tipo} | Região: {RA} ({endereco}) | Preço: {faixa} (R$ {custo_medio}) |
Vibes: {vibes} | Experiência: {resumo_sensorial} | TikTok: {transcricao_whisper_500c}
```

3. Estratégia de Embeddings e Banco Vetorial

A indexação semântica e a recuperação densa foram implementadas em `src/rag/banco_vetorial.py` com as seguintes especificações técnicas:

- **Modelo de Embedding:** `text-embedding-3-large` (OpenAI) / `gemini-embedding-001`.
- **Dimensionalidade: 3.072 dimensões** (vetores densos de alta fidelidade sem truncamento).
- **Justificativa Técnica:** Forte alinhamento semântico com expressões regionais e coloquiais de Brasília, associando intenções abstratas a atributos reais (ex.: associar *"passeio ao ar livre com criança gastando pouco"* a *"Eixão do Lazer / Kid Friendly / Gratuito"*).
- **Métrica de Busca e Ranking:** Similaridade de Cosseno calculada entre o vetor da consulta ($\vec{q}$) e os $N = 93$ vetores indexados ($\vec{d_i}$), retornando o Top-K ($K = 3$) de maior aderência:

$$\text{Similaridade}(\vec{q}, \vec{d_i}) = \frac{\vec{q} \cdot \vec{d_i}}{\|\vec{q}\| \, \|\vec{d_i}\|} = \frac{\sum_{j=1}^{3072} q_j \, d_{i,j}}{\sqrt{\sum_{j=1}^{3072} q_j^2} \sqrt{\sum_{j=1}^{3072} d_{i,j}^2}}$$

- **Persistência e Portabilidade:** Os 93 vetores estão persistidos em `data/banco_vetorial_embeddings.json` (5.9 MB), permitindo carregamento em memória com latência de busca sub-milissegundo.

4. Pipeline de Ingestão de Dados (Curadoria em 2 Fatores — 2FA)

Para impedir que vídeos patrocinados fora de escopo ou estabelecimentos sem endereço/preço contaminassem o RAG, foi construído o pipeline de **Autenticação de Dois Fatores de Dados (2FA)** em `src/rag/coletor_tiktok_2fa.py`:

Etapa 1 — Coleta & ASR	**164 vídeos brutos (2026)** de `@rolezandoembrasilia` → Áudio extraído (`yt-dlp`) → Transcrição via Groq (`whisper-large-v3-turbo`).
Etapa 2 — FATOR 1 (Cultural)	**Filtro Semântico via LLM:** Avalia se o vídeo retrata local real de gastronomia/lazer. • *Reprovados (47 vídeos):* Propaganda de varejo/serviços descartada automaticamente. • *Aprovados (117 vídeos):* Extração estruturada de Nome, Categoria e Vibes.
Etapa 3 — FATOR 2 (Prático)	**Dupla Trava Geográfica e Financeira:** Exige Endereço no DF (via GPS) + Preço (R$). • *Incompletos (24 vídeos):* Retidos na quarentena (`data/fila_quarentena_2fa.json`). • *Aprovados (**93 fontes**):* Promovidos à Base Curada (`data/banco_aprovado_2fa.json`).
Etapa 4 — Indexação Vetorial	**Chunking & Embeddings:** Montagem dos 93 chunks → Geração de vetores 3.072-d → Persistência em `data/banco_vetorial_embeddings.json`.

5. Arquitetura do Sistema e Telemetria em Tempo de Execução

O fluxo de execução (*Runtime RAG*) opera integrado ao SDK do **Langfuse Cloud**, registrando cada etapa de recuperação:

1. Consulta & Vetorização	2. Busca Vetorial (Top-3)	3. Geração Fundamentada
Pergunta do usuário vetorizada em 3.072 dimensões.	Cálculo de Cosseno contra os 93 vetores curados, retornando o Top-3 mais aderente.	Síntese fundamentada nos chunks, citando fontes e links diretos do TikTok.

Camada de Observabilidade (Langfuse Cloud): Rastreamento de spans com cálculo e envio dos scores de governança: `fidelidade_rnf03 = 1.0`, `sla_latencia_rnf14 <= 2.5s` e controle de tokens/custo por requisição.

6. Telemetria, Observabilidade e Auditoria de RNFs (Langfuse Cloud)

A instrumentação em `src/telemetria/telemetria.py` audita o pipeline a cada requisição na API ou na interface web:

1. `fidelidade_rnf03` **(Meta: Zero Alucinação) — Score Obtido: 1.0 (100% ancorado):** Todos os estabelecimentos recomendados pertencem estritamente aos 93 documentos homologados em `data/banco_aprovado_2fa.json`.
2. `sla_latencia_rnf14` **(Meta ≤ 2.5 s) — Score Obtido: 1.0 (Conforme):** Latência de processamento e recuperação vetorial entre **2.8 ms e 21.9 ms**, assegurando fluidez instantânea na resposta.
3. `qualidade_recuperacao_rag` **(Relevância Semântica) — Score Obtido: 0.87 a 0.98:** Alta precisão no cruzamento entre a intenção do usuário e os dados auditados.

6.1. Casos de Teste Auditados com Traces Rastreáveis

ID	Consulta Submetida	1º Recuperado	Região	Cosseno	Latência	Trace ID (Langfuse)
#1	*"Qual o melhor boteco alternativo na Asa Sul?"*	**Spelunca DF**	Asa Sul	**0.98**	21.9 ms	`ac2ece1889e84b72`
#2	*"Quero um café romântico e calmo na Asa Norte"*	**Café e Um Chêro**	Asa Norte	**0.98**	5.5 ms	`2fe411a5e04f46ab`
#3	*"Opção ao ar livre, baratinha e kid friendly"*	**Eixão do Lazer**	Eixo Rodov.	**0.87**	2.8 ms	`0a0f08b4188045f1`

6.2. Análise Técnica dos Resultados

- **Desambiguação Geográfica e de Categoria (Caso #1):** Filtrou bares de outras regiões e cafeterias, selecionando o *Spelunca DF* pelo cruzamento de categoria (`Bar/Pub`), vibe (`Alternativo`) e região (`Asa Sul`).
- **Captura de Nuances de Atmosfera (Caso #2):** A transcrição Whisper preservou o clima intimista e tranquilo para conversa a dois, alcançando similaridade 0.98 para o *Café e Um Chêro*.
- **Combinação Multifatorial (Caso #3):** A busca cruzou espaço aberto, custo acessível e ambiente para crianças, ranqueando o *Eixão do Lazer* em 1º lugar.

6.3. Governança de Custos e Consumo de Tokens (RNF-15)

- **Consumo Médio por Consulta:** ~ 280 tokens de entrada e ~ 65 tokens de saída.
- **Custo Estimado por Requisição: $0.000045 USD** com Gemini Flash, permitindo mais de 30.000 consultas mensais na cota sem custos extras.

7. Demonstração e Como Testar o Protótipo

7.1. Opção 1: Demonstração Online Direta no Navegador (Recomendada)

Acesse a aplicação publicada em nuvem: **https://orquestrador-roles.onrender.com/rag** ou explore todos os dados interativamente em **/entregaveis**.

7.2. Opção 2: Bateria de Testes via Terminal

Na raiz do repositório, execute o comando abaixo (ou via container Docker):

```
python3 src/rag/prototipo_rag.py
```

O script executa os **4 passos obrigatórios da avaliação**: **(1)** Consulta submetida; **(2)** Top-3 Chunks recuperados com escores de cosseno; **(3)** Resposta fundamentada com fontes; e **(4)** Evidência completa das fontes (ID, GPS, preço, transcrição e link do TikTok). O log completo está em `docs/entregas/semana-02/DEMONSTRACAO_RAG_ENTREGAVEL.txt`.

8. Preparação para a Semana 3 (Baseline LLM Pura vs. RAG)

Conforme o cronograma pedagógico do Challenge 2, a **Semana 3** terá como foco a comparação empírica entre respostas sem contexto (*LLM pura / Zero-Shot*) e respostas ancoradas (*RAG fundamentado em evidências*). A base de 93 rolês homologados no 2FA já estabelece a **linha de base quantitativa (*Baseline*)**.

8.1. Metodologia do Benchmark Comparativo (*Golden Dataset*)

Bateria padronizada de prompts cobrindo especificidades do DF:

1. **Consultas de Nicho Local:** Estabelecimentos específicos de Brasília para quantificar a taxa em que a LLM pura inventa quadras ou indica locais fechados.
2. **Restrições de Orçamento e Horário:** Avaliação dos valores em R$ e filtros de funcionamento (ex.: opções gratuitas no domingo ou até R$ 35,00 em Taguatinga).
3. **Avaliação Automatizada no Langfuse (*LLM-as-a-Judge*):** Pontuação par a par (*LLM Pura* vs. *RAG*) quanto a Fidelidade Factual, Precisão Geoespacial e Rastreabilidade de Fontes.

8.2. Matriz de Tratamento de Casos de Borda e Guardrails

Cenário de Borda	Comportamento do Buscador	Mecanismo de Salvaguarda (Guardrail)
Consulta Fora de Escopo *(ex.: "Jantar no Leblon")*	Similaridade máxima abaixo do limiar de corte (Score < 0.65).	O sistema não alucina e avisa que o escopo é estrito ao Distrito Federal.
Restrições Conflitantes *(ex.: "Parrilla nobre grátis")*	Recupera semântica de alta gastronomia, mas inspeciona `faixa_preco`.	Explicita a faixa real (`$$$`) e sugere opções acessíveis coerentes.
Gírias e Expressões do DF *(ex.: "rolê no eixão")*	Vetor de 3.072 dimensões captura expressões coloquiais transcritas pelo Whisper.	Recuperação correta e robusta mesmo com termos regionais de Brasília.

9. Conclusão e Links de Acesso ao Ecossistema

A entrega da Semana 2 cumpre integralmente todos os critérios técnicos do Segundo Desafio: **volume superado** (93 fontes contra o mínimo de 20), **chunking semântico autocontido**, **vetores densos de 3.072 dimensões**, **geração 100% ancorada com fontes**, **observabilidade no Langfuse Cloud** e **sustentabilidade operacional**.

Recurso do Projeto	Ambiente / Tipo	Endereço Direto / Link de Acesso
Central de Entregáveis Interativa	Produção (Render)	`https://orquestrador-roles.onrender.com/entregaveis`
Interface do RAG com Telemetria	Produção (Render)	`https://orquestrador-roles.onrender.com/rag`
Aplicação Web Principal	Produção (Render)	`https://orquestrador-roles.onrender.com`
Log de Demonstração (4 Passos)	Repositório Git	`docs/entregas/semana-02/DEMONSTRACAO_RAG_ENTREGAVEL.txt`
Pacote Compactado Oficial	Arquivo ZIP	`docs/entregas/semana-02/Entregavel_Grupo7_Base_Conhecimento_e_RAG.zip`